

D.3.1. Midterm Project Status Report

COMP.8157: Advanced Database Topics (2026S) — Group # 4

Project name/ID	Performance Evaluation of Vector Indexing Techniques for AI Search Systems
Project manager	Group 4 (rotating lead by phase)
Date	July 3, 2026
Team members	Devu Babu Sheeja, Ashwin Senthur Pandian, Rishi Gaurangkumar Patel, Nikhil Goud Nathi

Overall Project Health

On Track	Preprocessing, embeddings, benchmarking automation, visualization, and the Flat/IVF indexes are done and have already produced real results on the dataset. HNSW and the tuning sweeps are the pieces still in progress, and they are on schedule for the remaining weeks.
----------	--

This project is building a benchmarking framework that compares three vector search indexes, Flat Search, IVF, and HNSW, under the same conditions using FAISS and Sentence-BERT embeddings on the ABC News Headlines dataset. The goal is to measure query latency, retrieval accuracy (recall@K), memory usage, index build time, and how each method scales as the dataset grows, and to see which index is actually the better choice and under what conditions. So far the team has finished the full data and embedding pipeline, the automated benchmarking and visualization system, and two of the three indexes (Flat and IVF), and has already run real experiments on twenty thousand real news headlines plus a scalability test from one thousand to one hundred thousand vectors. HNSW and the parameter-tuning experiments are what's left, and the team is on track to have those done well before the final demo and report deadlines.

1. Project Progress Breakdown

The table below breaks the project into the tasks the team has been working through, in the order they had to happen (each task depends on the ones before it in the pipeline: data, then embeddings, then indexes, then querying and evaluation, then visualization and reporting). Eleven of sixteen tasks are done, which the team feels is solid, visible progress since the project started.

ID	Task Description	Owner	Status	Remarks / Dependencies
1	Picked and justified the dataset (ABC News Headlines, ~1.24M rows, CC0 licence)	Devu	Completed	Chosen for size, topic variety, and open licence
2	Built the data ingestion and preprocessing pipeline (data_ingestion.py, prepare_headlines.py)	Devu	Completed	Depends on T1
3	Set up deterministic Sentence-BERT embedding generation (embeddings.py, 384-d)	Devu	Completed	Depends on T2
4	Prepared scaled corpora for scalability testing (1K to 100K entries)	Devu	Completed	Depends on T2
5	Built the config-driven benchmark runner (run_benchmark.py, YAML configs)	Nikhil	Completed	Ties T3 and the indexing layer together
6	Implemented Flat Search index (exact search, ground-truth baseline)	Ashwin	Completed	Depends on T3
7	Implemented IVF index (k-means clustering, nprobe search parameter)	Ashwin	Completed	Depends on T3
8	Built the query engine (per-query timing, recall@K vs. Flat)	Rishi	Completed	Depends on T6 for ground truth
9	Built benchmark orchestration for single-run and scalability modes	Rishi	Completed	Depends on T5, T8
10	Ran first benchmarks on Flat and IVF (real headlines + 1K-100K scale test)	Rishi	Completed	Depends on T6, T7, T9

ID	Task Description	Owner	Status	Remarks / Dependencies
11	Built the visualization system (charts + result tables)	Nikhil	Completed	Depends on T9
12	HNSW index (graph-based search)	Ashwin	In Progress	Depends on T3
13	Parameter-tuning sweeps (nprobe, efSearch)	Ashwin	In Progress	Depends on T7, T12
14	Full HNSW benchmarking and tuning results	Rishi	Remaining	Depends on T12, T13
15	Final results write-up and report	Team	Remaining	Depends on T14
16	Final demo and in-class presentation	Team	Remaining	Due July 26-28

So far the team has hit every milestone on time: the pre-proposal on May 17, the full proposal and its recorded presentation on May 31 and June 2, and now this midterm report.

Milestone Review

Milestone	Target Date	Status
D.1 Pre-proposal submitted	May 17	Complete
D.2.1 Full proposal + recorded presentation	May 31 / Jun 2	Complete
D.3.1 Midterm status report (this document)	Jun 28 (reported Jul 3)	Complete
HNSW + tuning sweeps complete	Mid-July (target)	Upcoming
D.4/D.5 Final demo + report	Jul 26	Not started
D.6 In-class presentation	Jul 28	Not started

2. GitHub Repository and Version Control Evidence

The team is using GitHub to track development, with tasks also organized on a GitHub Projects board split into the four ownership areas listed below. The repository link is:

Repository: <https://github.com/nikhilnathi2003/vector-index-benchmark>

The repository currently holds only the parts of the project that are marked Completed in Section 1: the data and preprocessing scripts, the embedding module, the Flat and IVF indexes, the query engine and benchmark runner, the visualization code, the experiment configs, and the README. Each member uploads their own completed work under their own GitHub account so the commit history reflects who actually did it, rather than one person pushing everything. The HNSW index and the parameter-tuning script are intentionally left out for now, since they are still marked In Progress and have not been fully tested; they will be pushed once they pass testing, at which point the repository will match the Completed column in Section 1 in full.

Work is split into four areas with no overlap between them, matching the task ownership in Section 1. **Devu Babu Sheeja** handles data and embeddings: `data_ingestion.py`, `prepare_headlines.py`, and `embeddings.py`, plus picking and justifying the dataset and preparing the scaled corpora. **Ashwin Senthur Pandian** handles the indexes: Flat and IVF are done in `indexing.py`, and HNSW plus the tuning sweeps in `tune_parameters.py` are in progress. **Rishi Gaurangkumar Patel** handles querying and evaluation: `query_engine.py` for timing and `recall@K`, and `benchmark.py` for running everything consistently and pulling the results together. **Nikhil Goud Nathi** handles the experiment runner and visualization: `run_benchmark.py`, `visualization.py`, the config files, and set up the repository and project board.

3. Demonstration of Completed Work

The team can demo a working version of the pipeline live. Running one command, `python run_benchmark.py` with a chosen config file, loads the corpus, generates the embeddings, builds the Flat and IVF indexes, runs two hundred test queries against each with timing, checks `recall@10` against the Flat ground truth, and outputs a results table with charts. Everything shown in the demo matches the Completed items in Section 1, nothing more.

The numbers from these runs so far: on twenty thousand real news headlines, Flat Search takes about 1.33 ms per query with perfect recall (1.000), since it checks everything, while IVF takes about 0.19 ms, roughly seven times faster, and still gets recall of 1.000 at basically the same memory usage. In the scalability test (1,000 to 100,000 vectors), Flat's query time grows in a straight line as the data grows, from about 0.08 ms up to about 7.4 ms, which is what you'd expect since it checks every vector, while IVF stays fast, around 1.2 ms even at 100,000 vectors, and keeps recall at 1.000 the whole way. One thing the team ran into and fixed along the way: at small dataset sizes IVF's clustering step can ask for more clusters than the data can support, so the code now scales the cluster count down automatically for small datasets, which avoided incorrect results at the smaller test sizes.

The main thing that makes this project useful rather than just re-running FAISS examples is that all three methods are tested on the exact same data, same embeddings, same queries, and same machine, so any difference in the results is actually caused by the indexing method and not by inconsistent test conditions, which is something the team noticed was missing from a lot of the papers and tutorials looked at earlier in the semester. The team is aware Flat Search gets slow at large scale, so it's only being used as the accuracy baseline, not as a method being proposed for production use. The team is also aware that IVF and HNSW's results depend a lot on their tuning parameters, which is exactly why the tuning experiment (T13) is planned, so the report doesn't just show one setting per method but the actual trade-off curve.

Team Progress

Tasks completed: data ingestion and preprocessing pipeline; dataset selection and justification; deterministic Sentence-BERT embedding generation; scaled corpora for scalability testing (1K-100K); config-driven benchmark runner; Flat Search and IVF indexes; query engine with per-query timing and recall@K; benchmark orchestration for single and scalability runs; first real-data and scalability benchmarks; and the visualization system for charts and result tables.

Tasks planned for the next period: finish and test the HNSW index; build and run the parameter-tuning sweeps for IVF's nprobe and HNSW's efSearch; push the remaining code to the repository once tested; run the full scalability and tuning experiments across all three methods; and begin writing the final report and preparing the presentation.

Risks and Action Plans

Category	Item	Action Plan
Known issue	At small dataset sizes, IVF's requested cluster count (nlist) could exceed what the data can actually support, which FAISS flagged with a training warning.	Fixed: the code now scales nlist down automatically based on dataset size, so IVF stays valid and warning-free from 1,000 to 100,000 vectors.
Known issue	An early benchmarking run showed inconsistent recall for IVF between two runs of the same configuration.	Traced to differences in how the corpus was being sampled between runs; fixed by seeding all random sampling, and confirmed fixed by re-running and comparing results.
Potential risk	Flat Search becomes very slow at large scale (over 7 ms per query at 100,000 vectors), which could make later large-scale experiments slow to run.	Flat is only used as the accuracy ground truth, not as a method being proposed for use, so this is expected and budgeted for in the experiment run time.
Potential risk	HNSW and the tuning sweeps are the last major implementation pieces and are currently in progress with about four weeks left before the final deadlines.	The team is prioritizing finishing HNSW and tuning early in the next period to leave enough time for full results analysis and report writing before July 26.
Change request	None at this time.	The approach approved in the full proposal is still being followed as planned; no scope changes have been requested.

4. Environment and Tools

The project is implemented in Python 3.13 using FAISS for the index structures (Flat and IVF now, HNSW in progress), the sentence-transformers library with the all-MiniLM-L6-v2 model for embeddings, and NumPy, Pandas, and Matplotlib for data handling and the result charts. Experiments are defined in YAML configuration files so a run is fully reproducible from the command line without editing code, and randomness (data sampling,

query selection) is seeded so repeated runs of the same configuration produce the same results. This matters for the project's goal specifically, since the whole point is comparing indexing methods fairly, and that comparison only means something if every run is set up the same way every time. Development is being done locally on Windows by each member and pushed to the shared GitHub repository referenced in Section 2, with the GitHub Projects board used to track which of the sixteen tasks in Section 1 each person is currently working on.

Summary

Overall, the team is on schedule and has more built and tested than the midterm checkpoint strictly requires: two full indexing methods are implemented, benchmarked on real data, and validated for correctness against an exact ground truth, and the entire pipeline runs end to end from a single command with reproducible, seeded experiments. The remaining work, HNSW, the parameter-tuning sweeps, the full results write-up, and the final presentation, is well scoped and the team does not currently see anything blocking it from being finished comfortably before the July 26 and July 28 deadlines.